Examen A5, M2 SETI

Numéro d'anonymat :

Durée 1h30

Répondre sur les feuilles du sujet aux questions suivantes (l'espace réservé à chaque question est largement suffisant pour une réponse juste) :

A- Codage - Bus

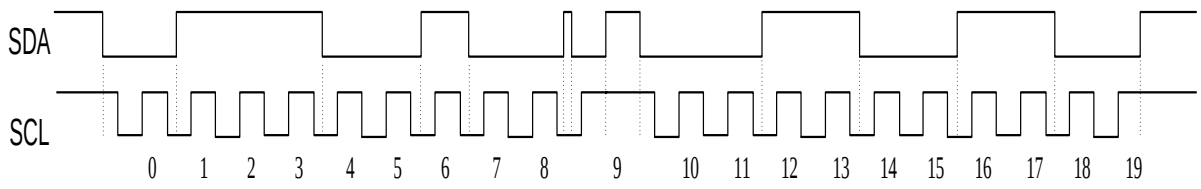
1. Donner, pour l'un des codages suivants, le principe ainsi qu'un exemple d'application : NRZI ou Manchester.

2. On souhaite transmettre des données d'un calculateur A vers un calculateur B, en utilisant une ligne série respectant le protocole RS232 (1 bit de Start, 8 bits de données et 1 bit de Stop). Le calculateur A doit envoyer 40 octets de données toutes les 70 ms. Le module de transmission RS232 accepte des vitesses de transferts suivantes : 2400 Bauds, 4800 Bauds, 9600 Bauds, 28800 Bauds. Chaque bit transmis sur la ligne doit être maintenu pendant au moins 100µs. Déduire la vitesse de transmission optimale qui respecte les contraintes de temps imposées. Toute réponse devra être justifiée.

3. Quelle est la différence entre les deux liaisons RS232 et RS485 ?

4. Donner les protocoles de gestion du flux dans une liaison série.

5. On dispose d'un chronogramme d'échange de données sur un bus I2C composé d'un microcontrôleur et de plusieurs périphériques esclaves, tous reliés sur ce bus I2C. Le microcontrôleur est le maître du bus. Identifiez sur le chronogramme ci-dessous les adresses des esclaves sollicités et précisez l'action demandée sur les périphériques (Lecture ou Ecriture). Précisez si les esclaves sollicités sont présents sur le bus. Justifiez votre réponse.



6. Le bus SPI est moins rapide que le bus I2C, Vrai ou faut ?

7. Peut-on avoir plusieurs Maîtres sur un bus I2C? si oui, comment fonctionne alors l'arbitrage?

8. Expliquez le principe de la gestion des interruptions, avantages par rapport au mode « scrutation ou Polling » ?

B- Interfaces

1- Quelle est la différence entre deux fonctions l'une synchrone et l'autre asynchrone ?

2- Quel est l'avantage de la programmation événementielle ?

C- QCM :

Répondre aux propositions, en cochant la bonne case du QCM suivant :

		VRAI	FAUX
a)	La gestion de périphérique par interruptions consiste à scruter périodiquement ce périphérique pour vérifier s'il est prêt à fonctionner.		
b)	Un contrôleur DMA, est un périphérique permettant de superviser les échanges de données qui s'effectuent entre le microcontrôleur et sa mémoire de données.		
c)	Une mémoire double accès est une mémoire permettant à un microcontrôleur d'accéder 2 fois de suite à cette mémoire.		
d)	Le bit de STUFFING (ou bourrage) est inséré pour gérer la priorité des émissions.		
e)	Le mécanisme de filtrage CAN (registres AC et AM) du µC 80C592 est n'est capable d'accepter qu'un seul identifiant unique, codé sur 11 bits.		
f)	Le code CRC de la trame CAN permet de vérifier si deux trames n'ont pas été émises en même temps.		
g)	Une transmission série RS232 est compatible bus CAN si on réalise un envoi de tous les champs constituant d'une trame CAN.		
h)	Chaque nœud interfacé à un bus CAN dispose d'une adresse physique configurable.		
i)	Lorsque deux trames CAN sont envoyées en même temps sur le BUS CAN, il y a collision et les deux trames sont perdues.		
j)	On dit qu'un système est "temps réel" lorsqu'il traite les données avec une fréquence la plus grande possible		

D- Bus de terrain CAN

Un calculateur CP à base de microcontrôleur 80C592 est utilisé pour contrôler un système de production. Ce calculateur et tous les capteurs et actionneurs sont reliés entre eux par un bus CAN 2.0A (à 1Mbits/s).

On s'intéresse au programme implanté sur le calculateur de contrôle (CP) dont une partie du code source (en C) est donné en annexe 1. Pour simplifier sa lisibilité, on remplacera par des pointillés (…) les parties de code C non intéressante pour notre étude.

On suppose que le contrôleur CAN du 80C592 est bien configuré et que les registres Acceptance_Code (noté AC) et Acceptance_Mask (noté AM) sont initialisés à :

AC= 0x52 et AM= 0x30

1) A partir des registres AC et AM, indiquez les plages d'identifiants (codés sur 11 bits) des trames qui seront acceptées par le module de filtrage matériel CAN.

En analysant le programme donné en annexe on s'aperçoit qu'un filtrage supplémentaire logiciel est réalisé sur les identifiants des trames acceptés par le filtrage matériel. D'autres part, le calculateur CP envoi des trames CAN en réponse à certaines trames reçues.

2) Déduire du programme les identifiants (codés sur 11 bits) des trames CAN reçues et qui seront traitées par l'application.

3) Déduire les identifiants (codées sur 11 bits) des trames CAN reçues qui donneront lieu à l'envoi d'un message sur ce même bus CAN.

4) Au niveau de l'instruction multi-choix ***switch(id)*** du programme, certains choix sur l'identifiant ne seront jamais exécutés. Expliquez pourquoi en justifiant votre réponse.

Annexe 1: code source C du programme contrôleur

```
……
// tableau prédéfini des trames à émettre
const unsigned char TTX[4][10]={    {0x64 , 0x83 , 0x34 , 0x21 , 0x44 , 0 , 0 , 0 , 0 , 0},
                                    {0x36 , 0x42 , 0x30 , 0x01 , 0 , 0 , 0 , 0 , 0 , 0},
                                    {0x58 , 0x22 , 0x00 , 0x11 , 0 , 0 , 0 , 0 , 0 , 0},
                                    {0x69 , 0x43 , 0xFE , 0xAB , 0x22 , 0 , 0 , 0 , 0 , 0}
                                };
// num_mess est l'indice pour choisir quel message envoyer
unsigned char dlc, num_mess=0;
unsigned int id;        // variable pour calculer l'identifiant de la trame reçue
/*********************************************************************/
```

```
/*      Réception des données par le bus CAN et traitement                    */
/*************************************************************************/

void    main()
{
        //Initialisation du CAN
        CANInitialisation();
        .......
        while(1)
        {
                // besoin d'une initialisation si erreur
                if  ( can_init_request || can_system_error ){
                        CANInitialisation( );
                        can_init_request=0;
                }
                if ( can_rx_ready )    // condition vraie si le filtrage accepte la trame
                {
                        can_rx_ready=0;
                        //déduire l'identifiant de la trame recue
                        id= ((unsigned int) tab_buffrx[0] <<3)| (tab_buffrx[1] >>5);
                        // retrouver la taille de données de la trame
                        dlc= tab_buffrx[1]&0x0F;
                        // traitement en fonction de l'identifiant de la trame recue
                        switch (id) {
                                case 0x213:
                                        .........
                                        num_mess=1;
                                        break;
                                case 0x341:
                                        .........
                                        break;
                                case 0x295:
                                        .........
                                        num_mess=3;
                                        break;
                                case 0x292:
                                        .........
                                        break;
                                case 0x383:
                                        .........
                                        break;
                                case 0x215:
                                        .........
                                        num_mess=2;
                                        break;
                                default:
                                        break;
                        }  // fin du switch
                }

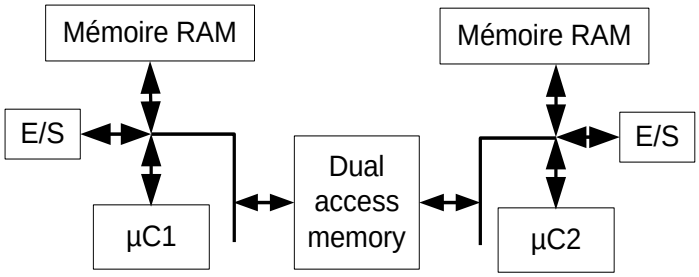

                if(can_tx_ready && (num_mess !=0) ){
                        //Emission des trames .....
                        // la trame à émettre dépend de l'identifiant num_mess utilisé comme index
                        send_message(TTX[num_mess]);
                        num_mess = 0; // trame émise num_mess initialisé
                        ......
                }

        }  //accolade de la boucle while(1)

}  // fin du programme main
```

E- Échange de données

On souhaite étudier un système informatique utilisant 2 microcontrôleurs (identiques comme ceux utilisés en TPs). Ces derniers s'échangent des données au travers d'une mémoire partagée. La figure ci-contre donne une représentation de l'architecture. La mémoire double accès (Dual Access), les E/S et les mémoires sont organisées en Octets. La mémoire double accès et les E/S sont vues dans l'espace d'adressage mémoire externe des microcontrôleurs.



On considère que le µC1 capture des données des E/S vers sa mémoire privée (RAM) et ensuite envoi le bloc de données au µC2, par la mémoire double accès (DualRam). Le microcontrôleur µC2 effectue la lecture de cette mémoire DualRam en transférant le bloc de données dans sa mémoire privée (RAM) pour ensuite traiter ces données. On utilisera une variable **M** (déclarée **char M**) pour synchroniser les échanges entre les deux microcontrôleurs (µC1 et µC2). L'algorithme de chaque processeur µC1 et µC2 sera le suivant :

Algorithme d'écriture par le µC1 :	Algorithme de lecture par µC2 :

While(1) {
Acquisition du bloc de données ;
Attendre que M passe à 0;
Ecrire données de RAM vers DualRAM
M=1;
}
	While(1) {
	Attendre que M passe à 1;
	Transférer données DualRAM vers RAM;
	Traiter les données en RAM ;
	M=0;
	}

1) Précisez où devra être implantée la variable **M** ?

On considère que le bloc de données qui est traité par µC1, puis transféré vers la mémoire double accès (DualRam), occupe 64 octets. Ce même bloc sera lu par µC2, transféré de DualRam vers sa mémoire privée.

Le temps d'acquisition du bloc de données sur µC1 (vers sa mémoire RAM) est de 5ms.

Le temps de traitement du bloc de données sur µC2 (dans sa mémoire RAM) est de 4ms.

IMPORTANT :

- Un **cycle d'instruction** du microcontrôleur nécessite **12 cycles d'horloge**. La Fréquence de l'horloge du microcontrôleur est de **12Mhz**.
- Le transfert, par logiciel, **d'un octet** de mémoire à mémoire nécessite **11cycles d'instruction,** c'est à dire 11µs.

2) Déduire la période d'acquisition d'un bloc de données par le µC1.

3) Pensez-vous que ce système soit viable ? Justifiez votre réponse quantitativement.

On souhaite améliorer le système. Compte tenu du temps occupé par le transfert de données de la RAM du µC1 vers la DualRAM et de DualRam vers la RAM du µC2, on décide d'utiliser un contrôleur DMA sur chacun des microcontrôleurs µC1 et µC2. On négligera le temps nécessaire de configuration des registres du contrôleur DMA.

Le temps d'accès à la mémoire RAM (privée de chaque microcontrôleur) et à la mémoire DualRam est de **20ns** (en lecture ou en écriture). Les microcontrôleurs (µC1 et µC2) libèrent le bus d'accès à la mémoire externe pendant 4 **périodes d'horloges** sur 12 **périodes d'horloges** (donc 4 périodes d'horloge par cycle instruction). Le contrôleur DMA pourra prendre le contrôle du bus du microcontrôleur pendant ces 4 périodes d'horloges libres.

4) Indiquez le nombre maximum d'octets que pourra transférer le DMA par cycle instruction (c'est à dire une lecture suivi d'une écriture vers la mémoire).

5) En combien de **cycles d'instructions** sera transféré la totalité du bloc de données (64 octets) de DualRam vers RAM, ou inversement.

6) Déduire la nouvelle période d'acquisition d'un bloc de données par le µC1.

On souhaite améliorer, encore, les performances de la période d'acquisition d'un bloc de données par le µC1. On décide d'éliminer les transferts mémoires, RAM vers DualRam (sur µC1) et DualRam vers RAM (sur µC2). On décide d'utiliser une mémoire DualRam suffisamment grande pour cela.

7) Proposez une solution pour assurer l'acquisition de données (par µC1) et leur traitement (par µC2) sans utiliser de mémoire privée RAM sur µC1 et µC2.

8) Déduire la nouvelle période d'acquisition d'un bloc de données par le µC1.